Vantrex: A Context-Aware IoT Cyber Activity Detection System

Project Report

Author: Aseel Almutairi
Dataset: TON-IoT Network Dataset
Task: Binary Classification — Normal vs. Attack
Primary Metric: F1-Score

1. Introduction

IoT devices are widely used in homes, healthcare systems, industries, and smart cities. However, these devices often lack built-in security mechanisms, making them vulnerable to cyberattacks such as backdoors, DDoS, scanning, password attacks, ransomware, and man-in-the-middle attacks.

Vantrex is a machine learning system designed to detect abnormal behavior in IoT network traffic. It analyzes network traffic features, device telemetry, and contextual metadata to classify network events as either Normal or Attack. The system was trained using the TON-IoT Network Dataset and deployed through a Streamlit-based web application, where users can manually enter feature values or upload CSV files for prediction with confidence scores and risk levels.

Automated detection is essential because IoT environments generate millions of events daily, making manual monitoring impossible. Missed attacks can lead to data breaches, ransomware incidents, and infrastructure damage.

F1-score was selected because the dataset is imbalanced. A model predicting only Attack could achieve approximately 76% accuracy without meaningful learning. F1-score addresses this limitation by balancing Precision and Recall.

2. Dataset Description

Dataset: TON-IoT Network Dataset | File: data/train_test_network.csv

Property	Value
Total Rows	211,043
Duplicate Rows	~20,569
Rows After Deduplication	~190,474
Target Column	label (0=Normal, 1=Attack)
Normal Records	~50,000 (24%)
Attack Records	~161,000 (76%)

The dataset contains an approximate 3:1 imbalance between Attack and Normal classes, making accuracy insufficient as the primary evaluation metric.

Dataset Challenges

Missing values: No actual NaN values; missing values appeared as '-' strings.
Duplicate records: ~20,569 duplicates removed before train-test splitting.
Leakage features removed: src_ip, dst_ip, type, dns_query, ssl_subject, ssl_issuer, http_uri, http_user_agent, weird_addl

Feature Preprocessing (34 features):
Numeric: SimpleImputer(median) + StandardScaler()
Categorical: SimpleImputer(constant) + OneHotEncoder(handle_unknown='ignore')
ColumnTransformer fitted only on training data to prevent leakage.

3. Models Used

Three machine learning models were trained and evaluated: Random Forest (F1 = 0.9991), Decision Tree (F1 = 0.9983), and Logistic Regression (F1 = 0.9910). Random Forest achieved the highest F1-score and showed the strongest overall performance.

4. Neural Network

My network had three Dense layers (128, 64, and 32 neurons) with Dropout regularization after the first two layers. During training, the model showed no significant overfitting and achieved an F1-score of 0.9964, showing strong performance, although Random Forest achieved the best overall results.

NLP Assistant

A lightweight rule-based NLP assistant was integrated into the Streamlit application to help users understand project outputs. It uses basic text preprocessing and keyword-based intent matching to answer project-related questions.

5. Results and Comparison

All models were trained on 80% of the dataset and evaluated on the remaining 20%.

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Random Forest	0.9985	0.9987	0.9993	0.9990	0.9999
Decision Tree	0.9970	0.9977	0.9985	0.9981	0.9937
Neural Network	0.9944	0.9954	0.9975	0.9964	0.9993
Logistic Regression	0.9861	0.9943	0.9879	0.9911	0.9951

Metric Priority for Intrusion Detection

1. Recall - minimize missed attacks
2. Precision - reduce false alarms
3. F1-score - balance both metrics

Best Model: Random Forest

- Highest F1-score (0.9990) and Recall (0.9993)
- ROC-AUC of 0.9999
- Faster inference and simpler deployment than Neural Network
- Final model saved as: models/best_pipeline.pkl

6.What I learned

1. **Data cleaning is critical**
   Duplicate records and placeholder values ('-') significantly affected preprocessing and evaluation quality.
2. **F1-score is more appropriate than accuracy for imbalanced datasets**
   High accuracy alone may hide poor performance on minority classes.
3. **Training and deployment preprocessing must remain identical**
   Saving the entire sklearn Pipeline prevents preprocessing mismatches.
4. **Complex models are not always better**
   Random Forest outperformed the Neural Network while remaining simpler and faster.
5. **Data leakage can artificially inflate results**
   Features such as attack type and IP addresses were removed to ensure realistic learning.
6. **Class imbalance requires deliberate handling**
   Using class_weight='balanced' improved model fairness across classes.

Future Work

- Multi-class attack type classification as a separate supervised learning task
- Cloud deployment through Streamlit Community Cloud
- Real-time packet capture integration